Using C# components in T3000

The shell is C++, and not everything around it is. This is how a C# component is added to a project that is built as C++.

Project Name: T3000_CrossPlatform

This document describes how to use a COM component written in C# , in the main C++/MFC application. Specifically, the control basic editor, it is done in C# and can be called from within the T3000

GITHUB: https://github.com/temcocontrols/T3000_CrossPlatform

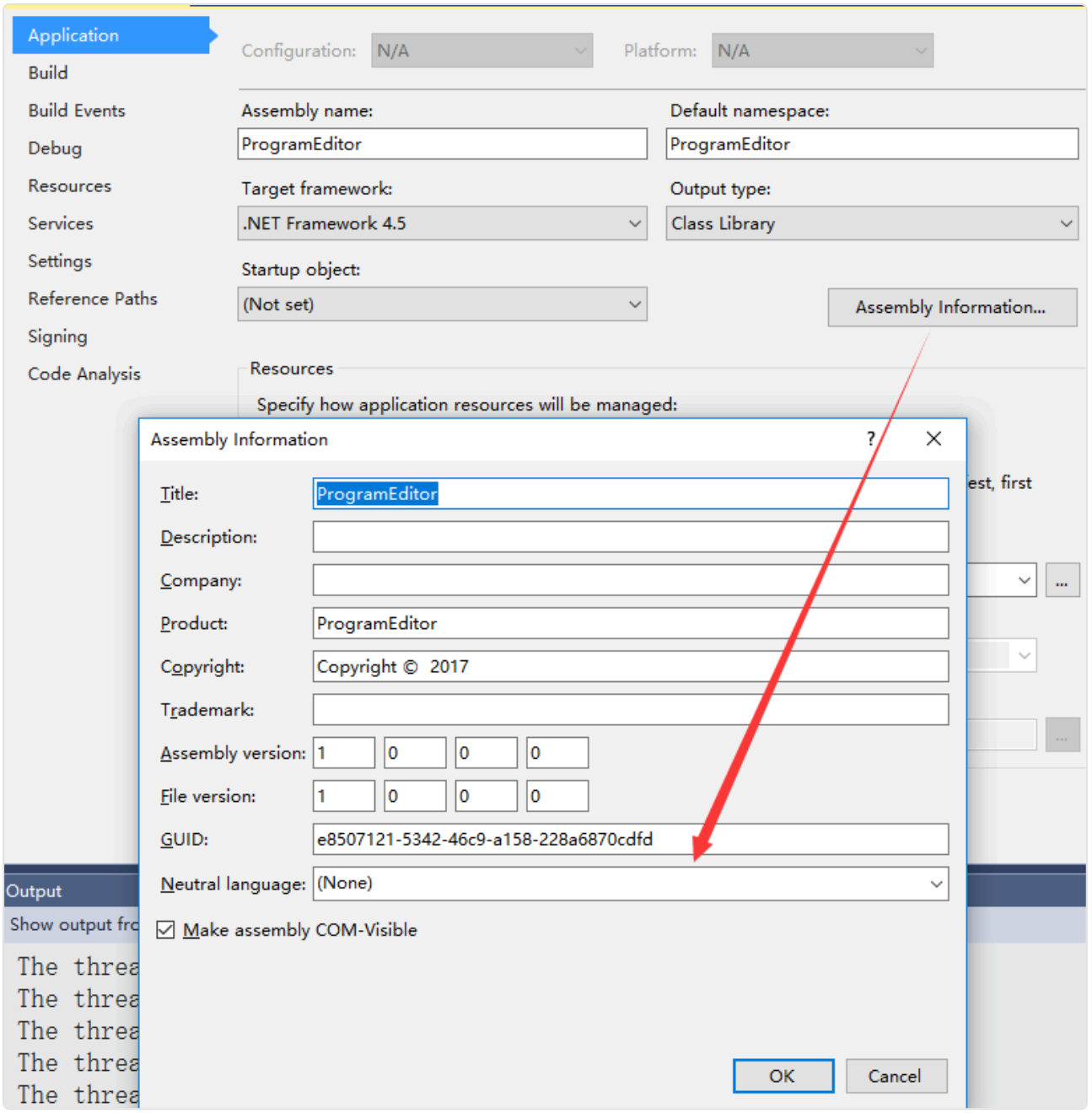
Here is a document about program editor

https://github.com/temcocontrols/T3000_CrossPlatform/tree/master/T3000Documenter/Help

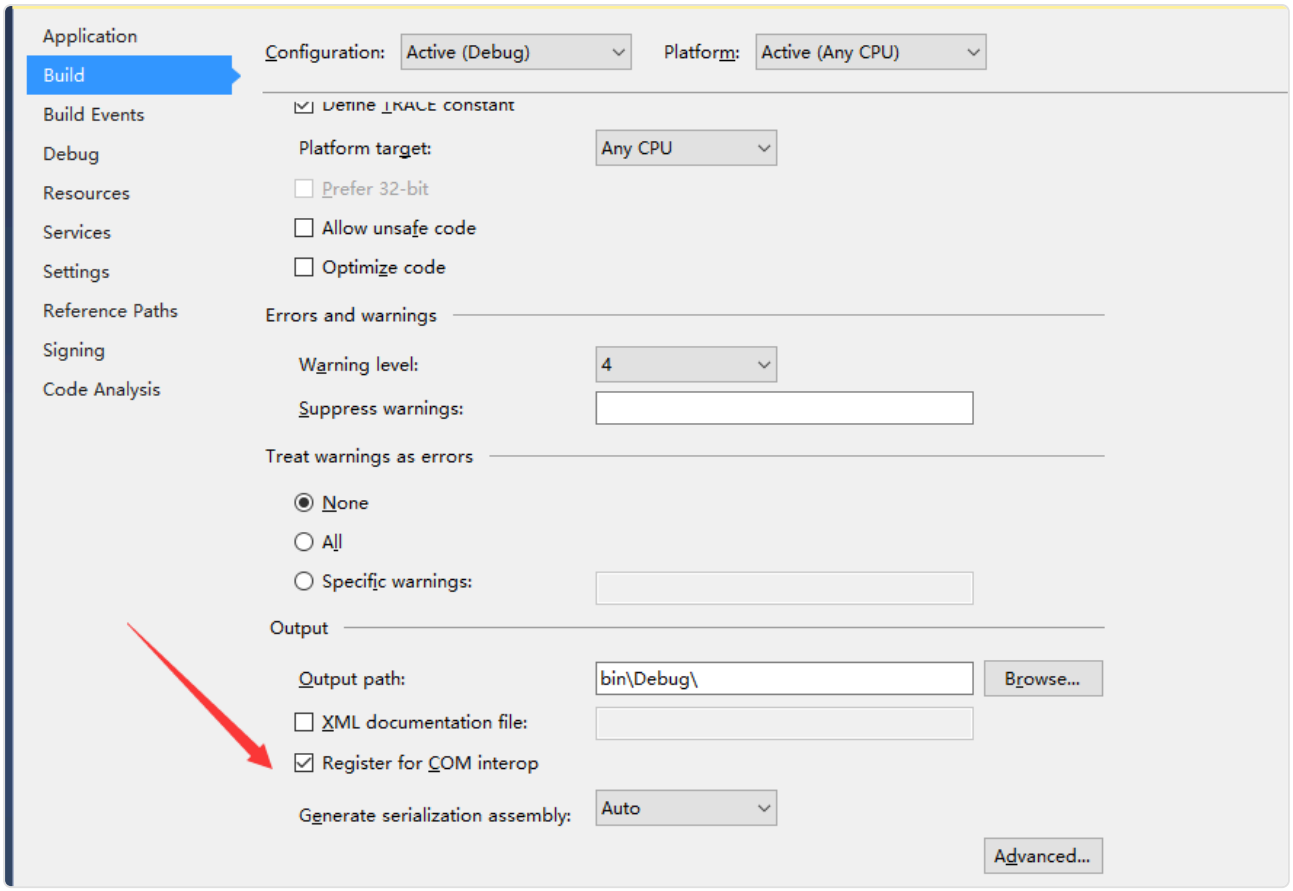
TBD: Move this to the \Documentation folder of the cross platform project.

How to Make C# Component use in MFC

Make the com visible

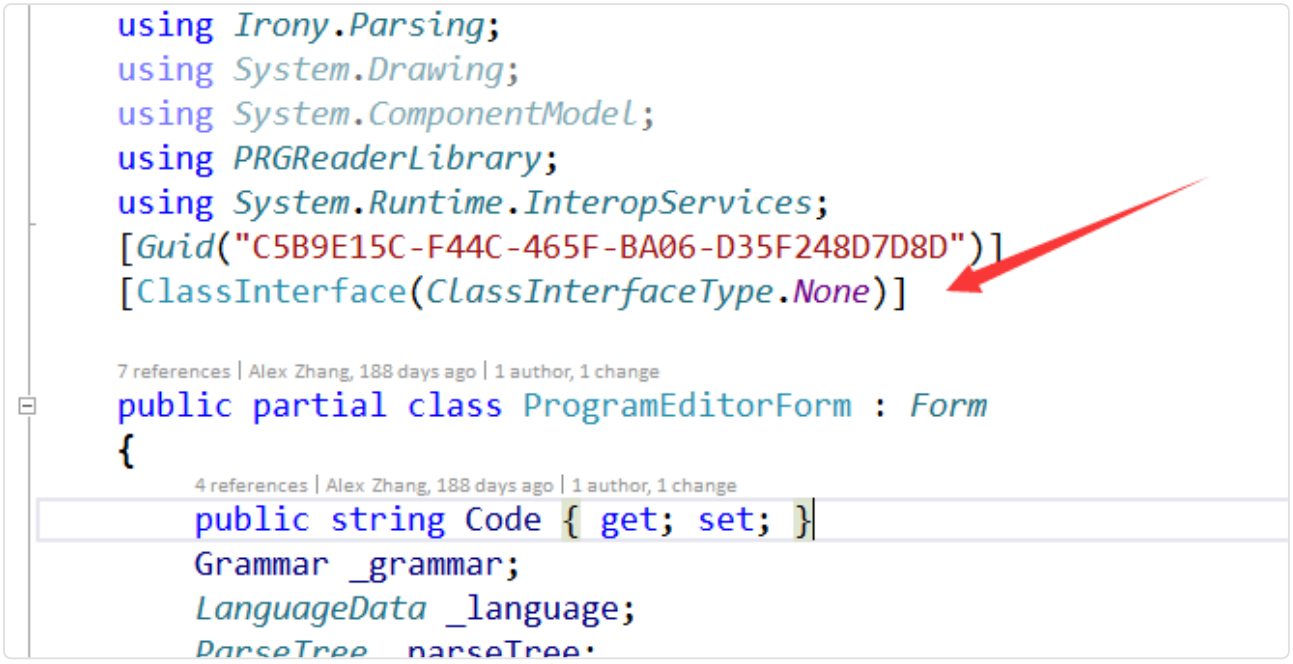


2.Register for COM interop



Put the ComUtilities class into the project .the code is in the background in the document .

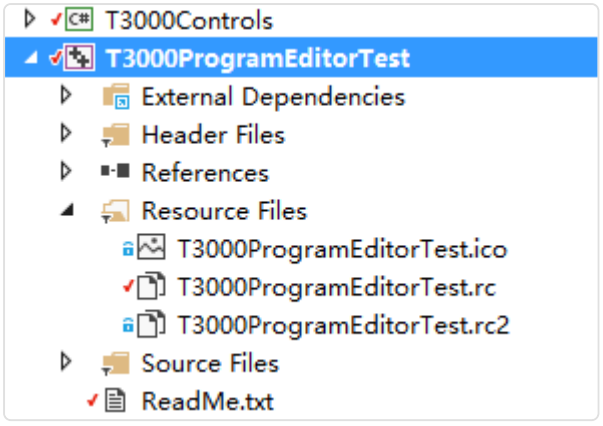
Select the component what you want to use in other language .

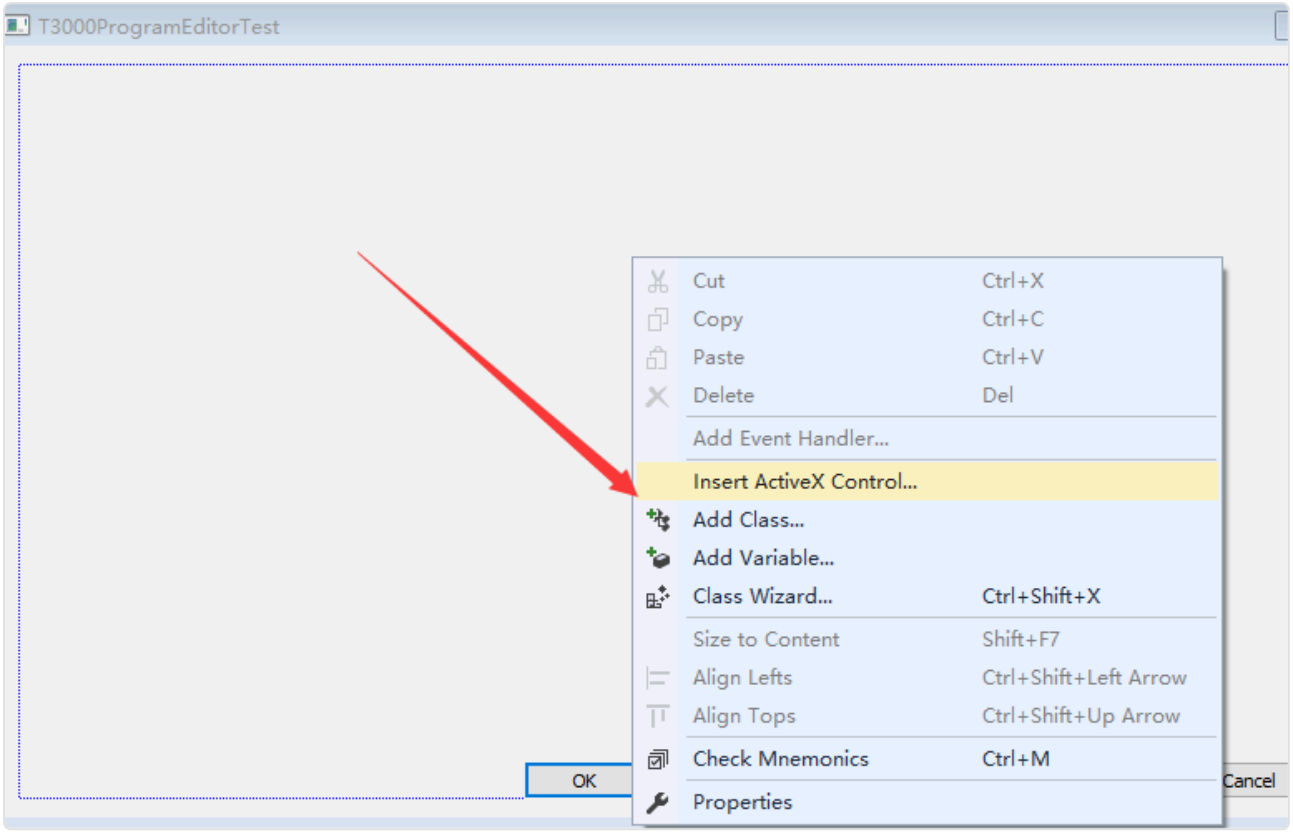


```
[ComRegisterFunction()]
0 references | 0 changes | 0 authors, 0 changes
public static void RegisterClass(Type type) => ComUtilities.RegisterControlClass(type);

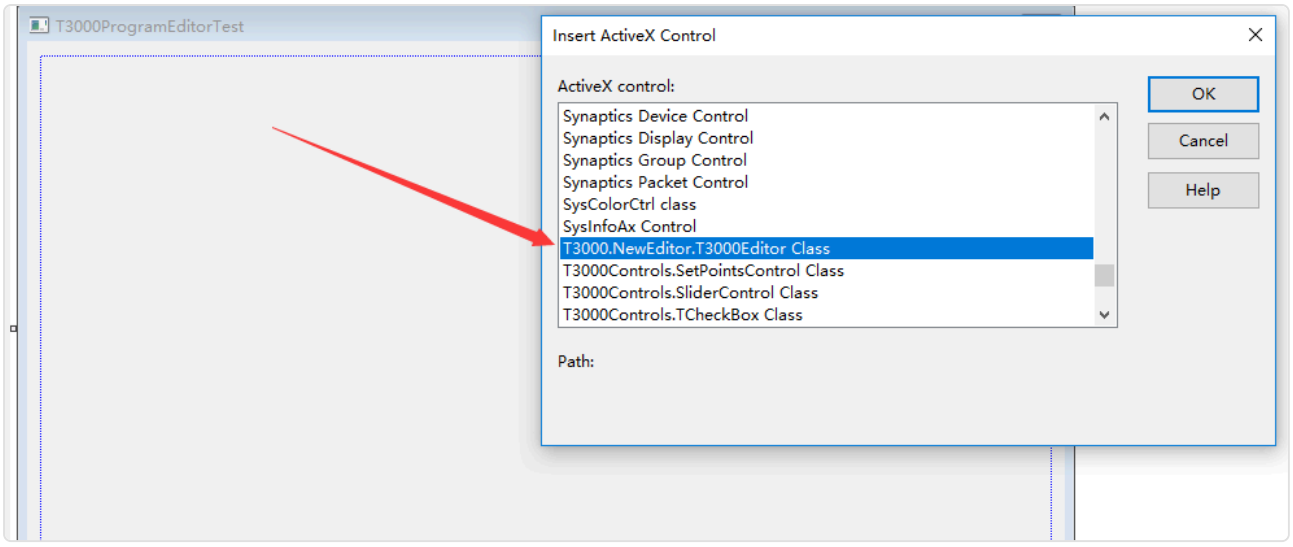
[ComUnregisterFunction()]
0 references | 0 changes | 0 authors, 0 changes
public static void UnregisterClass(Type type) => ComUtilities.UnregisterControlClass(type);
```

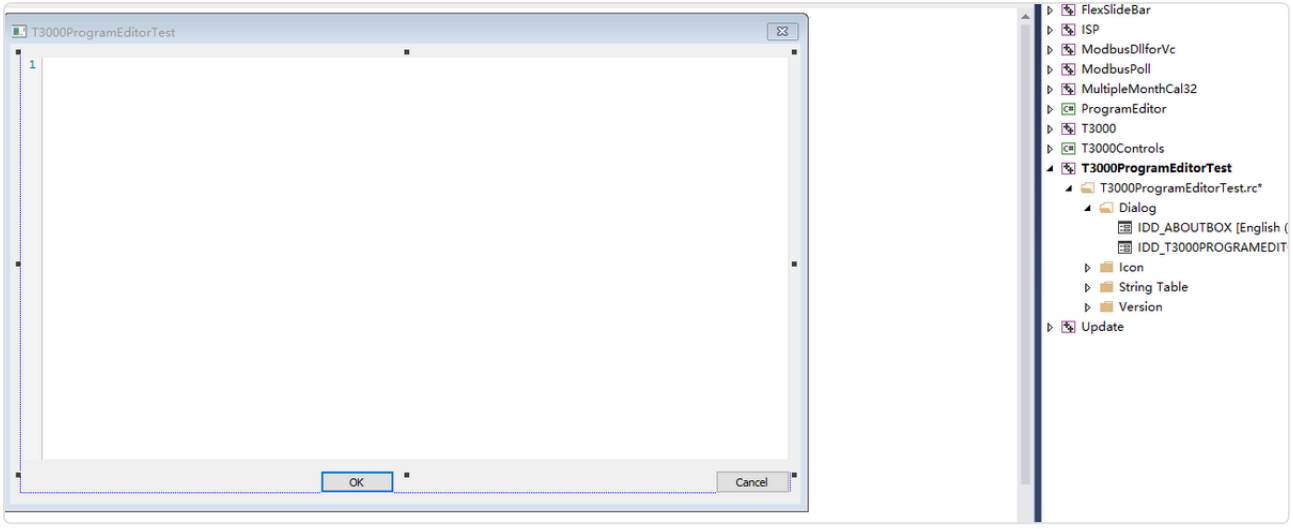
Make a test project



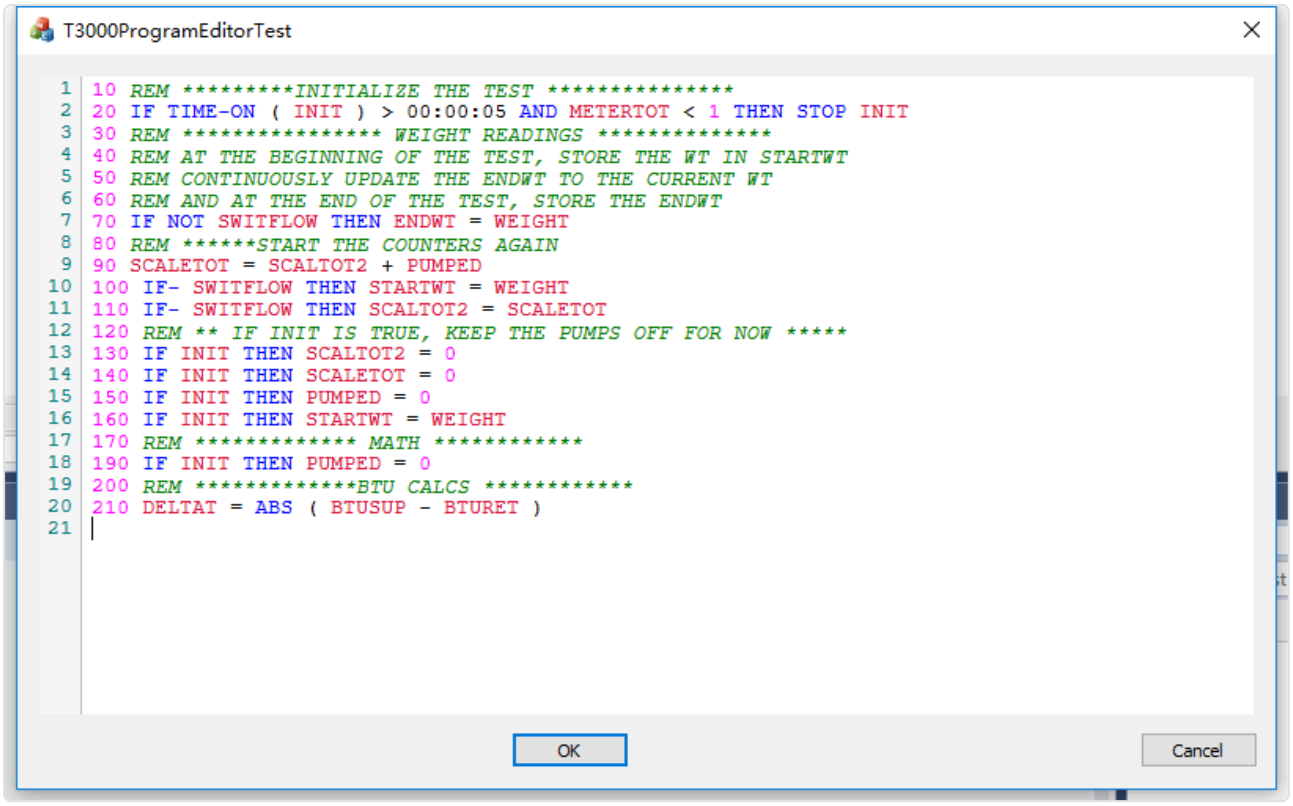


Select the component that we want to use .





F5 ,run the test project .



CODE – ComUtilities

```
namespace T3000.Forms

{

using System;

using System.Reflection;

using Microsoft.Win32;

using System.Runtime.InteropServices;

internal class NativeMethods

{

internal static class OLEMISC

{

internal const int OLEMISC_RECOMPOSEONRESIZE = 0x1;

internal const int OLEMISC_ONLYICONIC = 0x2;

internal const int OLEMISC_INSERTNOTREPLACE = 0x4;

internal const int OLEMISC_STATIC = 0x8;
```

```
internal const int OLEMISC_CANTLINKINSIDE = 0x10;

internal const int OLEMISC_CANLINKBYOLE1 = 0x20;

internal const int OLEMISC_ISLINKOBJECT = 0x40;

internal const int OLEMISC_INSIDEOUT = 0x80;

internal const int OLEMISC_ACTIVATEWHENVISIBLE = 0x100;

internal const int OLEMISC_RENDERINGISDEVICEINDEPENDENT = 0x200;

internal const int OLEMISC_INVISIBLEATRUNTIME = 0x400;

internal const int OLEMISC_ALWAYSRUN = 0x800;

internal const int OLEMISC_ACTSLIKEBUTTON = 0x1000;

internal const int OLEMISC_ACTSLIKELABEL = 0x2000;

internal const int OLEMISC_NOUIACTIVATE = 0x4000;

internal const int OLEMISC_ALIGNABLE = 0x8000;

internal const int OLEMISC_SIMPLEFRAME = 0x10000;

internal const int OLEMISC_SETCLIENTSITEFIRST = 0x20000;

internal const int OLEMISC_IMEMODE = 0x40000;

internal const int OLEMISC_IGNOREACTIVATEWHENVISIBLE = 0x80000;

internal const int OLEMISC_WANTSTOMENUMERGE = 0x100000;

internal const int OLEMISC_SUPPORTSMULTILEVELUNDO = 0x200000;

}

}

internal static class ComUtilities

{

///

/// Register the class as a control and set it's CodeBase entry.

/// Key format: "HKEY_CLASSES_ROOT\CLSID{guid}"

///
```

```
/// The type of the control
public static void RegisterControlClass(Type type, int bitmapId = 101)
{
var keyName = @"CLSID" + type.GUID.ToString("B");
using (var key = Registry.ClassesRoot.OpenSubKey(keyName, true))
{
key.SetValue(null, type.ToString() + " Class");
key.CreateSubKey("Programmable").Close();
key.CreateSubKey("Control").Close();
key.CreateSubKey("Insertable").Close();
using (var subkey = key.CreateSubKey("MiscStatus"))
{
var miscStatus =
NativeMethods.OLEMISC.OLEMISC_RECOMPOSEONRESIZE +
NativeMethods.OLEMISC.OLEMISC_INSIDEOUT +
NativeMethods.OLEMISC.OLEMISC_ACTIVATEWHENVISIBLE +
NativeMethods.OLEMISC.OLEMISC_SETCLIENTSITEFIRST;
subkey.SetValue(null, miscStatus.ToString());
}
using (var subkey = key.CreateSubKey("TypeLib"))
{
var libid = Marshal.GetTypeLibGuidForAssembly(type.Assembly);
subkey.SetValue(null, libid.ToString("B"));
}
//key.DeleteSubKeyTree("InprocServer32");
using (var subkey = key.CreateSubKey("InprocServer32"))
```

```
{
//subkey.SetValue(null, Environment.SystemDirectory + @"\mscoree.dll");
subkey.SetValue("CodeBase", Assembly.GetExecutingAssembly().CodeBase);
//subkey.SetValue(null, Assembly.GetExecutingAssembly().Location);
//subkey.SetValue("ThreadingModel", "Apartment");
}
//using (var subkey = key.CreateSubKey("Server"))
//{
// subkey.SetValue(null, "mscorld.dll");
//}
//
//using (var subkey = key.CreateSubKey("ToolboxBitmap32"))
//{
//subkey.SetValue(null,
// Assembly.GetExecutingAssembly().Location + ", " +
// bitmapId.ToString());
//}
using (var subkey = key.CreateSubKey("Implemented Categories"))
{
subkey.CreateSubKey("{7DD95801-9882-11CF-9FA9-00AA006C42C4}");
subkey.CreateSubKey("{7DD95802-9882-11CF-9FA9-00AA006C42C4}");
subkey.CreateSubKey("{40FC6ED3-2438-11CF-A3DB-080036F12502}");
subkey.CreateSubKey("{40FC6ED4-2438-11CF-A3DB-080036F12502}");
subkey.CreateSubKey("{40FC6ED5-2438-11CF-A3DB-080036F12502}");
subkey.CreateSubKey("{4FED769C-D8DB-44ea-99EA-65135757C156}");
subkey.CreateSubKey("{0DE86A52-2BAA-11CF-A229-00AA003D7352}");
```

```
subkey.CreateSubKey("{0DE86A53-2BAA-11CF-A229-00AA003D7352}");
subkey.CreateSubKey("{0DE86A57-2BAA-11CF-A229-00AA003D7352}");
}
using (var subkey = key.CreateSubKey("Version"))
{
int majorVersion;
int minorVersion;
Marshal.GetTypeLibVersionForAssembly(
type.Assembly, out majorVersion, out minorVersion);
subkey.SetValue(null, $"{majorVersion}.{minorVersion}");
}
key.CreateSubKey("InstalledVersion").Close();
}
}
///
/// Called to unregister the control.
/// Key format: "HKEY_CLASSES_ROOT\CLSID{guid}"
///
/// The type of the control
public static void UnregisterControlClass(Type type)
{
Registry.ClassesRoot.DeleteSubKeyTree(@"CLSID" + type.GUID.ToString("B"), false);
}
}
}
```